

Tour Planner Project Protocol

Technical Decisions and Development Process

Project Architecture

The Tour Planner application was implemented as a client-server application consisting of:

- **Backend:** Spring Boot (Java)
- **Frontend:** Angular
- **Database:** PostgreSQL
- **Authentication:** JWT (JSON Web Token)
- **External Services:** OpenRouteService (Geocoding and Routing)
- **Map Rendering:** Leaflet with OpenStreetMap

A REST architecture was chosen because it provides a clean separation between frontend and backend and allows each component to evolve independently.

Database Design

The database consists of four primary entities:

- Users
- Tours
- Tour Logs
- Transportation Types

Each user owns multiple tours, while each tour may contain multiple tour logs.

Foreign key constraints together with cascading deletes ensure referential integrity. Passwords are stored only as BCrypt hashes and never in plaintext.

Distance values are stored in kilometers, while time values are stored in seconds.

Authentication

Authentication is implemented using JWT.

The workflow is:

1. User registers or logs in.
2. Backend validates credentials.
3. Backend generates a signed JWT containing the user's email.

4. Angular stores the token in local storage.
5. Every protected request automatically attaches the JWT through an HTTP interceptor.
6. Spring Security validates the JWT before executing protected endpoints.

This approach was selected because it provides stateless authentication without requiring server-side sessions.

Tour Creation

Creating a tour involves multiple systems.

The frontend first geocodes the entered addresses using the backend.

The backend forwards the request to OpenRouteService, retrieves the route, calculates distance, estimated duration & route geometry and stores these values together with the tour.

Only the generated route geometry is stored in the database, allowing the route to be rendered again without contacting the routing service.

Search Implementation

Two search mechanisms were implemented.

Tour Search

Tours can be searched by textual properties such as:

- tour name
- description
- locations

The repository performs a full-text LIKE search using JPQL.

Tour Log Search

Logs can be searched using

- comment
- rating
- difficulty
- distance
- duration

The Search Service combines both search results into one response object that is displayed by the frontend.

Ownership and Authorization

Every tour belongs to one user.

Whenever a user edits or deletes data, ownership is verified before the operation is executed.

Unauthorized modifications result in an `AccessDeniedException`.

This guarantees that users cannot manipulate tours or logs created by other users.

Route Visualization

Routes are displayed using Leaflet.

Instead of requesting the route every time a tour is opened, the stored route geometry is parsed into coordinates and rendered as a polyline.

Markers are automatically placed at the start and destination.

This reduces API usage and improves performance.

Import and Export

Tours together with their logs can be exported into JSON.

The exported data contains

- tour information
- transportation type
- stored route
- all associated logs

Import recreates the complete object graph while assigning the imported tours to the currently authenticated user.

Child Friendliness Calculation

A child friendliness score was implemented as a computed property.

It combines

- average difficulty
- average duration
- average distance

Each value is normalized and contributes equally to the final score.

This value is calculated dynamically and therefore is not stored inside the database.

Problems Encountered and Solutions

Security Applied to Every Request

After implementing JWT authentication, every HTTP request was automatically protected by Spring Security. This caused requests that should have remained publicly accessible (such as login, registration, and geocoding during development) to return authentication errors.

The solution was to configure Spring Security to explicitly allow unauthenticated access to the authentication endpoints while keeping all remaining endpoints protected. On the frontend, an HTTP interceptor was implemented that automatically attaches the JWT only after a successful login, ensuring authenticated requests work transparently.

Updating Tour Coordinates

Updating an existing tour initially did not update the route correctly. Although the textual information (name and description) was saved, changes to the start location, destination, or transportation profile were not reflected in the stored route.

The problem occurred because changing the coordinates requires requesting a completely new route from OpenRouteService. Simply updating the database fields was insufficient.

The solution was to detect whether any routing-related properties had changed. If the start location, destination, or transportation type is modified, the backend requests a new route from OpenRouteService, recalculates the distance and estimated travel time, and replaces the stored route geometry before saving the updated tour.

Synchronizing Route Display

During development, the map sometimes displayed outdated routes after editing a tour because the frontend continued using previously loaded route information.

This was solved by refreshing the selected tour after every successful update and redrawing the route on the Leaflet map. The route is parsed from the stored GeoJSON and rendered again together with updated start and destination markers.

UML Documentation

The project documentation contains:

- Use Case Diagram
- Wireframes
- Database Class Diagram
- Entity Class Diagram
- Sequence Diagram for Full-Text Search

These diagrams document the overall system structure, user interaction, and search workflow.

Unit Testing

The implemented unit tests focus on the application's business logic rather than framework functionality. The service layer contains most of the application's custom logic, making it the most suitable target for unit testing. Controllers mainly forward HTTP requests and responses, while Spring Boot and Spring Security are extensively tested by their respective frameworks.

Tour Service

The TourService tests focus on the application's calculated properties rather than CRUD operations. The tests verify:

- popularity calculation based on the number of associated tour logs,
- child friendliness calculation under ideal, average, and worst-case conditions,
- correct handling of tours without logs,
- normalization of the child friendliness score to the range between 0 and 1,
- correct averaging across multiple tour logs.

These calculations are part of the application's core business logic and are derived dynamically instead of being stored in the database. Testing them ensures that users always receive consistent and accurate statistics.

Tour Log Service

The TourLogService tests verify the complete lifecycle of tour logs, including:

- creating new tour logs,
- retrieving existing tour logs,
- updating complete and partial tour log information,
- deleting tour logs,
- handling invalid tour or log identifiers,
- searching tour logs.

These tests ensure that user-generated data is created, modified, and retrieved correctly while handling exceptional situations appropriately.

Authentication Service

The AuthService tests verify the authentication workflow, including:

- successful user registration,
- rejection of duplicate email addresses,

- password confirmation during registration,
- password hashing before storage,
- successful user login,
- rejection of invalid login credentials,
- generation of JWT authentication tokens after successful authentication.

Authentication is a security-critical component because every protected endpoint depends on correct user verification. These tests ensure that only valid users can obtain authentication tokens and that sensitive information, such as passwords, is handled securely.

JWT Service

The JwtService tests verify the correctness of the JSON Web Token implementation by testing:

- JWT generation,
- extraction of the username from a token,
- validation of tokens for the correct user,
- handling of expired tokens,
- correctness of the generated token structure.

Since JWTs are used to authenticate every protected request, ensuring their correctness is essential for the security and reliability of the application.

Why These Tests Were Chosen

The selected unit tests target the application's custom business logic and security mechanisms, as these components are most likely to contain implementation errors and have the greatest impact on application correctness. By focusing on the service layer, the tests remain independent of the Spring framework while providing high coverage of the application's functionality. The tested methods include calculations, validation logic, authentication, and data manipulation, all of which are essential to the correct operation of the Tour Planner.

Git History

<https://github.com/Z1rael/tour-planner-app>

Time Tracking

The intermediate submission accounted for 29 hours of work, primarily spent on frontend development, architectural design, and the implementation of supporting components and mock services.

Additional effort was required for the final project, including backend development, database integration, authentication, routing service integration, testing, debugging, and documentation.

Task	Time
general .css	5 h
user mock service	3 h
leaflet integration	2 h
restructuring html	1 h
architecture	6 h
mock services	2 h
mediators/facades	4 h
initial components	5 h
Total Intermediate	29 h
Backend implementation	13 h
Database design & persistence	6 h
Authentication & security (JWT)	7 h
Tour management functionality	5 h
Tour log functionality	5 h
Search functionality	4 h
Routing API integration	5 h
Bug fixing & troubleshooting	8 h
Unit testing	8 h
Documentation & final report	4 h
Final Total	94 h